Informe 1 · Pruebas y cobertura

Sistema integrado de eventos laborales (JML) · Andina Servicios S.A.S. (empresa ficticia, datos ficticios)
Politécnico Grancolombiano · Gerencia de Proyectos en Seguridad de la Información · Grupo D01
Auditoría del 24 al 25 de octubre de 2026 · rama `auditoria/entrega` · commit `610fa6d` · generado el 25/10/2026

1. Resumen

Indicador	Documentado por el proyecto	Línea base medida (12/10 y 24/10)	Final medido
Pruebas automáticas sin integración	1 600	1 618	**2 574** pasan, 2 omitidas con motivo, 1 `xfail` declarada
Cobertura de ramas, dominio	«99 %»	97,74 %	**97,74 %** (combinada con líneas: 99,43 %)
Cobertura de ramas, aplicación	«99 %»	94,4 %	**94,83 %** (combinada: 98,65 %)
Cobertura de ramas, 7 módulos de seguridad	—	84,48 % (ingreso)	**100 %** los siete
Mutación, dominio	93,6 %	—	**97,8 %**
Mutación, aplicación	nunca medida	—	**92,7 %** (primera medición: 77,8 %)
Mutación, módulos de seguridad	nunca medida	—	**96,1 %** (primera medición: 83,8 %)
Pruebas contra GLPI 11 real	30	30	**30 de 30**
Pruebas contra PostgreSQL 17	—	251 pasan, 18 omitidas	251 pasan
Recorrido por navegador (Playwright) por el WAF	4	4 de 4	4 de 4 en la primera corrida; **3 de 4 en las dos siguientes** (B-36)
Estabilidad	—	20 de 20 corridas sin fallos	20 de 20
Defectos sembrados detectados	—	—	**6 de 6**

La cifra «99 %» del proyecto es la métrica **combinada** de `coverage` (líneas y ramas juntas); las ramas puras son 97,74 % y 94,83 % (hallazgo informativo B-14). Ambas superan las compuertas reales (95 % y 90 %).

2. La suite por capa

Medida con `pytest --junitxml` sobre el árbol final (`evidencias/funcionales/ANEXO-A-JUNIT`, 108 s):

Capa	Pruebas	Qué verifica
`dominio`	820	Reglas puras: máquina de estados, tickets, auditoría encadenada, perfiles, permisos, cuentas
`aplicacion`	249	Casos de uso con dobles en memoria; atomicidad de la unidad de trabajo; **denegaciones, rastro de auditoría y detalles exactos** (nuevas)
`contratos`	228	Cada puerto con su doble y su adaptador real, sin cambiar la prueba
`escenarios`	75	ING, VAC, CDP, CSU, RET y EMG completos en memoria; **Anexo A** (nuevas)
`adaptadores`	439	Vistas, persistencia, GLPI, identidad, worker, infraestructura; **accesibilidad, identidad y middleware exactos** (nuevas)
`seguridad`	680	Autorización por celda, IDOR y CSRF, TOTP, soporte, filtro de logs, despliegue, caídas (nuevas)
`bdd`	83 (+2 omitidas)	13 características Gherkin en español (nuevas)
Total	**2 574**	

Fuera de esa suite, con el stack local arriba: `tests/integracion` (PostgreSQL 17: 251 pasan, 18 omitidas por requerir el Supabase real), GLPI 11 real (30) y Playwright (4).

Las dos omitidas son los escenarios de F7 `[PENDIENTE: F7]` (S-38): la captura con `tshark` de N-1 y la petición a la Data API de Supabase, ya medidas en el informe 3 y en el catálogo. La `xfail` es una sola y está registrada: el Auditor no puede leer solicitudes (S-39 / B-26).

3. Cobertura de ramas

`pytest --cov=jml --cov-branch` sobre las siete carpetas de pruebas, con las compuertas de `make verificar` (`--fail-under=95` en dominio y `--fail-under=90` en aplicación). Evidencia: `F9-COBERTURA-FINAL`.

Grupo	Ramas	Ramas %	Líneas	Combinado %
dominio	303/310	97,74	1451/1454	99,43

Grupo	Ramas	Ramas %	Líneas	Combinado %
aplicación	220/232	94,83	1317/1326	98,65
adaptadores	380/406	93,6	2295/2343	97,31
total	**903/948**	**95,25**	**5063/5123**	**98,27**
seguridad: ingreso	58/58	100	282/282	100
seguridad: permisos	32/32	100	103/103	100
seguridad: auditoría	30/30	100	138/138	100
seguridad: soportes	16/16	100	73/73	100
seguridad: filtro de logs	14/14	100	39/39	100
seguridad: CSV y vistas técnicas	8/8	100	92/92	100
seguridad: middleware	6/6	100	53/53	100

Hallazgo de la medición: el flujo de ingreso estaba en 84,48 % (las ramas de error de Supabase). Se añadieron las pruebas que faltaban y subió a 100 % (B-13, B-24).

4. Mutación: ¿las pruebas notan que algo cambió?

La cobertura dice qué líneas se ejecutaron; la mutación dice si las pruebas se dan cuenta de que el código cambió. `mutmut` altera el código (invierte una comparación, cambia una constante, borra un argumento) y exige que alguna prueba falle.

Objetivo	Herramienta	Mutantes	Muertos	%	Meta
dominio	mutmut 2.5.1 (Windows)	1 341	1 312	**97,8**	85
aplicación, 1.ª medición	mutmut 3.8.0 (contenedor Linux)	2 226	1 731	77,8	85
aplicación, 2.ª (más directorios de pruebas)	idem	2 226	1 853	83,2	85
aplicación, final	idem	2 226	2 063	**92,7**	85
seguridad, 1.ª	idem	512	429	83,8	85
seguridad, final	idem	512	492	**96,1**	85

La lección de la primera medición. `aplicacion` dio 77,8 % porque se midió solo con `tests/aplicacion` y `tests/escenarios`; las pruebas de los casos de uso de auditoría viven en `tests/adaptadores`. Con todos los directorios que ejercitan ese código, 83,2 %. El resto lo cerraron cinco archivos de pruebas nuevos:

Archivo	Qué fija
`tests/aplicacion/test_denegaciones.py`	La **acción** y el **motivo** de cada `NoAutorizado`/`DatoInvalido`. La acción alimenta el evento `acceso.denegado` (B-25)
`tests/aplicacion/test_rastro_exacto.py`	El rastro de auditoría completo de un ING y de un RET (clave por clave) y las transiciones de la bandeja
`tests/aplicacion/test_detalles.py`	Entidad y referencia de cada `NoEncontrado`, roles ordenados, fronteras de los contratos por vencer
`tests/adaptadores/test_identidad_exacta.py`	Lo que el adaptador de Supabase **envía** (cuerpo, cabeceras, ruta, códigos 429 y 5xx)
`tests/adaptadores/test_middleware_y_registro_exactos.py`	Proxy confiable, `X-Request-ID`, CSP, forma del log JSON

Lo que sobrevive y por qué. En aplicación quedan 163 supervivientes y en seguridad 20. En seguridad se revisaron uno por uno: son *equivalentes* (el método HTTP en minúsculas y el nombre de una cabecera con otra caja, que `httpx` y Django tratan igual; valores por defecto que dan el mismo resultado; relleno base64 que Python tolera) salvo **un** mensaje de error sin cubrir. Las 29 del dominio son las mismas 27 equivalentes ya clasificadas en `docs/spikes/mutacion.md` más 2 de anotaciones.

5. Gherkin ejecutable

13 características en español (`# language: es`) con `pytest-bdd` 9.0.0 (solo `dev`): ingreso, autorización, inmutabilidad (en memoria y en PostgreSQL), restringidos, cabeceras, borde, soporte PDF, cuentas derivadas, resiliencia, Supabase aislada, cadena de suministro y secretos y logs. Cada una tiene camino feliz o frontera **y** un caso adversarial. `test_trazabilidad.py` exige que ninguna etiqueta quede sin trazar y que cada amenaza T1 a T21 y cada N-1 a N-5 tenga un escenario.

6. Pruebas de seguridad nuevas (`tests/seguridad` y `tests/integracion`)

Prueba	Control	Rojo sin el control
`test_segundo_factor.py`	Un código TOTP ya usado no vuelve a servir; un desafío ajeno no entra	defecto «segundo uso»
`test_soporte.py`	PDF políglota, 20 MB exactos y +1, nombre con `../`, `\x00` y ancho cero	defecto «sin firma»
`test_matriz_authz.py`	440 celdas (22 rutas × 2 métodos × 10 roles), 403 frente a 404	defecto «sin decorador de rol»
`test_idor_csrf.py`	Respuesta idéntica a un id inexistente; asignación masiva ignorada; CSRF	—
`test_filtro_de_logs.py`	Propiedad de 1 000 casos: ningún valor sensible sobrevive al filtro	filtro roto (halló B-21)
`test_despliegue.py`	`check --deploy` sin avisos	—
`test_caidas.py`	Supabase y GLPI caídos: ninguna ruta protegida responde 200 ni filtra detalle	—
`test_concurrencia.py`	Dos decisiones simultáneas: una gana	sin `select_for_update`
`test_roles_bd.py`	15 pruebas con `SET ROLE`: dueño, `REVOKE`, disparadores, `PUBLIC`/anon	tres defectos sembrados

7. Defectos sembrados: ¿detecta la suite lo que debe?

Seis defectos, de uno en uno, con restauración byte a byte: SQL por f-string (`ruff S608`), quitar el `REVOKE`, apagar el disparador, quitar el decorador de rol, aceptar el `.pdf` renombrado y permitir el segundo uso del TOTP. **Los seis fueron detectados**, además de tres adicionales (sin RLS, filtro roto, sin bloqueo). Una lección: el primer intento del defecto 4 **no** se detectó porque la vista repite la comprobación en un segundo caso de uso; es defensa en profundidad real, y la matriz no distingue cuál de las dos falló.

8. Anexo A (Excel): 157 filas contrastadas con el sistema

`tests/escenarios/test_anexo_a.py` (64 pruebas) **lee las celdas del Excel** y las compara con el sistema: emite cada evento y comprueba prioridad, vencimiento y listas de verificación por cola; compara las plantillas de GLPI, los campos por evento y las matrices sembradas; y prueba que los pares de roles no prohibidos se aceptan. Resultado: 138 filas cumplen, 9 parcialmente, 10 no se pueden verificar en local y ninguna falla. Detalle en el informe 4 y en `anexo-a-matriz.csv`.

9. Estabilidad y repetibilidad

20 corridas consecutivas de la suite unitaria: 20 de 20 sin fallos, 1 632 pruebas cada una, **sin reintentos automáticos**. La primera tanda se invalidó porque edité un archivo a mitad de la corrida; se repitió con el árbol quieto (`F1-06-ESTABILIDAD-CONTAMINADA` se conserva como prueba de ello).

10. Lo que no se pudo medir

- Pruebas contra el **Supabase real** de autenticación: solo existe un servidor simulado (Supabase en solo lectura de catálogo).
- Cobertura de los adaptadores contra Azure (Key Vault, Blob).
- El recorrido Playwright es sensible al limitador del WAF (2 r/s): en dos de tres corridas el caso RET recibió un 429. No es un defecto del código probado sino del ritmo del navegador (B-36).